Math 6601 Independent Study Project

Jacob Hauck

May 16, 2025

Proposal

For my independent study project I would like to study one of the most popular methods for computing all the eigenvalues and eigenvectors of an $n \times n$ matrix A: the QR method [1].

Goals

I would like to achieve the following goals with this project.

1. Understand the theory of the QR method by writing a description of the algorithm and a proof of its convergence based on the discussion in our textbook [1], filling the gaps of the textbook and explaining the rationale and motivation of the method.
2. Create an implementation of the QR method in MATLAB.

Project Tasks

I plan to achieve the above goals through the following steps.

1. Describe background on the QR decomposition, the primary tool used in the QR method: what it is, and how it is computed. I will use our textbook [1] and also the textbook from Applied Matrix Analysis [2] as sources for my own description.
2. Learn how to reduce a matrix to Hessenberg form by using Householder transformations, based on the description in the textbook. I will write a description of the algorithm and implement it in MATLAB. This first step of the QR method reduces the computational complexity of one step of the QR method iteration from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$.
3. I may need to create my own implementation of the QR decomposition for a Hessenberg matrix (I don't know if the built-in MATLAB implementation will do this) in order to exploit the advantage listed above.
4. Describe the QR method and its motivation. Here I will draw on our textbook but fill in gaps by proving assertions that the textbook makes without proof and explaining the motivation of the method as I understand it.
5. Prove the convergence of the QR method. I will write my own version of the textbook's proof, again filling in gaps and expanding on things that the textbook glosses over.
6. Implement the QR method. With the fast QR decomposition and reduction to Hessenberg form already implemented, the QR method itself is actually pretty simple to implement. I will also devise some tests to verify the correctness of the implementation, probably using examples from the textbook and some random matrices, and I will evaluate the convergence rate of the method for these examples.

1 Introduction

Computing the eigenvalues and eigenvectors of a given $n \times n$ matrix A is a very important process. For example, we saw in our class that the condition number of a symmetric, positive-definite matrix is closely related to its eigenvalues. A naive approach to computing eigenvalues and eigenvectors would be to use the definition: determine the characteristic polynomial by directly computing $\det(A - I\lambda)$, then find the roots numerically, using, say Newton's method with deflation. This naive approach, however, falls apart when n is even slightly large, as the cost of computing the characteristic polynomial scales terribly in n.

Another idea is to use the fact that $A^n x_0 \to \lambda_1^n v_1 + \mathcal{O}(|\lambda_2|^n)$ as $n \to \infty$, where $|\lambda_1| > |\lambda_2|$ are the two largest-in-magnitude eigenvalues of A, and v_1 is a unit eigenvector corresponding to λ_1. By re-scaling appropriately, one obtains the *power method* (Section 5.2 in our book [1]), which is an iterative method for computing the largest eigenvalue of A and a corresponding eigenvector. This method only produces one eigen-pair, and only if A has a unique, largest, simple (having only one eigenvector) eigenvalue; it can be applied repeatedly with deflation to find the other eigenvalues of A, assuming the deflated matrices all have unique, largest, simple eigenvalues.

Using the easily-proved fact that λ is an eigenvalue of A if and only if $(\lambda_0 - \lambda)^{-1}$ is an eigenvalue of $(A - I\lambda_0)^{-1}$, one can then apply the power method to $(A - I\lambda_0)^{-1}$. By choosing λ_0 judiciously, one can ensure that $(A - I\lambda_0)^{-1}$ has a unique, largest, simple eigenvalue, then apply the power method to determine the eigenvalue. Back-transforming gives a corresponding eigenvalue λ of A. In this way any eigenvalue of A may be obtained by the right choice of λ_0, with a higher convergence rate if λ_0 is chosen close to λ. This is called the *inverse power method* (Section 5.3 in our book [1]); although it overcomes the main difficulty of the power method, it still requires choosing λ_0 properly, which is not trivial.

Rather than use the naive approach, power iteration, or inverse power iteration, modern linear algebra libraries use a method called the *QR method* (Section 5.5 in our book [1]). The QR method relies on iterated QR decompositions, which (with the right optimizations) can be much faster, more robust, and more stable than the above three methods and converges to a matrix that provides all the eigenvalues and eigenvectors of A in one iterative process, avoiding the need for deflation or repeated iterative processes used in the power methods[1].

2 Theory of the QR Method

2.1 Review of QR decomposition

First we recall the definition of QR decomposition.

Definition 1. *For a matrix $A \in \mathbb{C}^{n\times n}$, if $A = QR$, where Q is unitary and R is upper-triangular, the the product QR is called a* QR decomposition *of A.*

Note the careful wording here; QR decompositions are not unique. The QR decomposition is the main tool used in the QR method, and non-uniqueness of QR decompositions is an important issue in discussing convergence of the QR method. To this end, we prove a lemma that explains just how different two QR decompositions of the same matrix can be. As it turns out, they can differ only by a diagonal unitary factor, which ultimately does not interfere with the proof of the convergence of the QR method.

Lemma 1. *Let $A \in \mathbb{C}^{n\times n}$ be a nonsingular matrix, and let $A = Q_1R_1$ and $A = Q_2R_2$ be two QR decompositions of B. Then there exists a diagonal, unitary matrix M such that $Q_1 = Q_2M$ and $R_2 = MR_1$.*

Proof. Set $Q = Q_2^*Q_1$ and $R = R_2R_1^{-1}$. Then

$$Q = Q_2^*Q_1 = Q_2^*BR_1^{-1} = Q_2^*Q_2R_2R_1^{-1} = R_2R_1^{-1} = R.$$

[1]Power and inverse power iteration are still useful techniques in some circumstances; for example, to compute only the largest/smallest eigenvalues of a symmetric positive-definite matrix in order to compute the condition number of the matrix.

Then R is unitary and upper-triangular. Since R is upper-triangular we can write $R = \begin{bmatrix} S & a \\ 0 & r_{nn} \end{bmatrix}$, where S is the $(n-1) \times (n-1)$ upper-left block of R, and r_{kj} is the element of R in the kth row and jth column, so that

$$a = \begin{bmatrix} r_{1n} \\ r_{2n} \\ \vdots \\ r_{(n-1)n} \end{bmatrix}.$$

On the other hand, R is unitary, so $RR^* = I$, that is,

$$\begin{bmatrix} I & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} S & a \\ 0 & r_{nn} \end{bmatrix} \begin{bmatrix} S^* & 0 \\ a^* & \bar{r}_{nn} \end{bmatrix} = \begin{bmatrix} SS^* + aa^* & \bar{r}_{nn} a \\ r_{nn} a^* & |r_{nn}|^2 \end{bmatrix}.$$

Then $|r_{nn}| = 1$, and $\bar{r}_{nn} \neq 0$ implies that $a = 0$. This gives $SS^* = I$, so S is unitary and upper-triangular. Repeating this argument inductively shows that $R = Q$ is diagonal and unitary. Setting $M = R$, we have

$$Q_1 = Q_2 M, \qquad R_2 = M R_1.$$

□

Computing QR decompositions

We also recall the Householder method for computing the QR decomposition of a matrix $A \in \mathbb{C}^{n \times n}$ in Algorithm 1, which requires $\mathcal{O}(n^3)$ multiplications and additions. We do not derive this algorithm again here or count the operations, as we already did it in class and homework. The algorithm is still important here, as we will use it later to obtain the algorithm for QR decompositions of Hessenberg matrices.

Algorithm 1: QR Decomposition with Householder Reflectors

Input: $A \in \mathbb{C}^{n \times n}$
Output: $Q \in \mathbb{C}^{n \times n}$, a unitary matrix
Output: $R \in \mathbb{C}^{n \times n}$, an upper-triangular matrix such that $A = QR$

$Q \leftarrow I$; // I is $n \times n$ identity
$R \leftarrow A$;
for $k = 1, 2, \ldots, n-1$ **do**
  $u \leftarrow \begin{bmatrix} R_{kk} & R_{(k+1)k} & \cdots & R_{nk} \end{bmatrix}^T$;
  $\alpha = \mathrm{rot}(u_1)\|u\|$; // $\mathrm{rot}(u_1)$ gets a unit complex number rotated 180° from u_1
  $u_1 \leftarrow u_1 - \alpha$;
  $v \leftarrow \frac{u}{\|u\|}$;
  $R^{(k)} \leftarrow R^{(k)} - 2vv^* R^{(k)}$; // $R^{(k)}$ is the bottom-right $(n-k+1) \times (n-k+1)$ submatrix of R
  $Q^{(k)} \leftarrow Q^{(k)} - 2vv^* Q^{(k)}$; // $Q^{(k)}$ is the last $n-k+1$ rows of Q
end
$Q \leftarrow Q^*$; // We were actually tracking Q^* through the algorithm

2.2 Motivation and the basic algorithm

The core idea behind the QR method is the observation that similarity transformations preserve eigenvalues; that is, if P is an invertible matrix, then $B = PAP^{-1}$ has the same eigenvalues as A, and v is an eigenvector of A if and only if Pv is an eigenvector of B. By choosing P cleverly, it may be easier to find the eigenvalues and eigenvectors of B; then the eigenvalues of A are known immediately, and the eigenvectors can be found by applying P^{-1} to the eigenvectors of B, if desired.

Theorem 1. *If P is an invertible matrix, then $B = PAP^{-1}$ has the same eigenvalues as A, and v is an eigenvector of A if and only if Pv is an eigenvector of B.*

Proof. Let λ be an eigenvalue of A with corresponding eigenvector v. Then

$$BPv = PAP^{-1}Pv = PAv = P\lambda v = \lambda Pv,$$

so λ is an eigenvalue of B, and Pv is an eigenvector of B (note that $Pv \neq 0$ because $v \neq 0$ and P is invertible).

Conversely suppose that λ is an eigenvalue of B with corresponding eigenvector w. Then

$$A(P^{-1}w) = P^{-1}PAP^{-1}w = P^{-1}Bw = P^{-1}\lambda w = \lambda P^{-1}w,$$

so λ is an eigenvalue of A, and $P^{-1}w$ is an eigenvector of A (again, $P^{-1} \neq 0$ because P is invertible and $w \neq 0$). □

The basic QR method

The main question now is how to choose P. The idea of the QR method is to choose P to be a product of unitary[2] matrices obtained through QR decompositions, with the hope of transforming A through similarity transformations into and triangular matrix B, whose eigenvalues and eigenvectors are easy to compute—note that the upper-triangular requirement seems to fit nicely with the upper triangular matrices that will also be obtained from QR decomposition, and the fact that P needs to be inverted is trivial since we will be constructing it to be unitary. The use of unitary similarity transforms has the nice benefit that it makes the algorithm as a whole more numerically stable than power iteration.

The fact that every matrix is unitarily similar to an upper-triangular matrix by the Schur decomposition theorem—see section 7.2 in Meyer's book [2]—gives us some hope that the above procedure is possible. In fact, the goal of the QR method is essentially to find an (approximate) Schur decomposition of A.

How, then, do we choose P? The simplest idea is just to choose $P = Q$. Then we obtain the following transformation:

$$A \mapsto QAQ^{-1} = QQRQ^*,$$

as $A = QR$. This suggests that $P = Q^{-1} = Q^*$ might work better, as it will cause a cancellation. Then we obtain the basic similarity transformation underlying the QR method:

$$A \mapsto Q^*AQ = Q^*QRQ = RQ. \tag{1}$$

Iterating this transformation yields the basic QR method algorithm, which is described precisely in Algorithm 2. Note that a key requirement for this algorithm to work is that the eigenvalues be *separated*; that is, if $\lambda_1, \ldots, \lambda_n$ are the eigenvalues of A, then

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|.$$

The basic algorithm may not work without this assumption, as we will see in our numerical experiments and the following theoretical investigation. We will justify the convergence criterion used to stop Algorithm 2 from the convergence theory of the basic method, to which we now turn.

Remark 1. Separation of eigenvalues implies that A is diagonalizable (by Jordan decomposition and the uniqueness of the eigenvalues; see section 7.8 of Meyer's book [2]); thus each eigenvalue has a distinct eigenvector, and the eigenvectors are linearly independent.

[2] One can also consider orthogonal matrices if A is a real matrix, but we begin by focusing on the complex case, as it easier to explain.

Remark 2. The eigenvectors of the upper-triangular matrix that results from QR iteration can be computed in $\mathcal{O}(n^2)$ operations (because of the upper-triangular structure), which is roughly the same cost as just one iteration of the QR method.

Remark 3. The basic QR method is often considered to be most suitable for symmetric positive-definite matrices [1], as the separation of eigenvalues is easier to guarantee, and real arithmetic can be used instead of complex (if the matrix is real) because the eigenvalues and eigenvectors are all real. Additionally, the iteration actually approaches a diagonal matrix if A is symmetric, making the eigenvectors trivial to compute.

Algorithm 2: The Basic QR Method

Input: $A \in \mathbb{C}^{n\times n}$, a matrix with separated, nonzero eigenvalues
Input: $\varepsilon > 0$, stopping tolerance
Output: $\lambda_1, \lambda_2, \ldots, \lambda_n \in \mathbb{C}$, approximate eigenvalues of A
Output: $v_1, v_2, \ldots, v_n \in \mathbb{C}^n$, approximate eigenvectors of A corresponding to $\lambda_1, \ldots, \lambda_n$

$A_1 \leftarrow A$;
$k \leftarrow 1$;
repeat
  $Q_k, R_k \leftarrow \mathrm{qr}(A_k)$; `// qr is QR decomposition`
  $A_{k+1} \leftarrow R_k Q_k$;
  $k \leftarrow k + 1$;
until $|[A_k]_{j\ell}| < \varepsilon$ for $j > \ell$;
for $i = 1, 2, \ldots, n$ **do**
  $\lambda_i \leftarrow [A_k]_{ii}$; `// [A_k]_ii is the ith diagonal element of A`
  $w_i \leftarrow$ eigenvector of A_k corresponding to λ_i;
  $v_i \leftarrow Q_1 Q_2 \cdots Q_{k-1} w_i$; `// Back-transformation from Theorem 1`
end

Convergence of the basic QR method

The convergence of the basic QR method is tied directly to the convergence of power iteration; indeed, the basic QR method can essentially be viewed as a faster, stabler implementation of power iteration with "automatic" deflation, as we will see. Define

$$\widetilde{Q}_k = Q_1 Q_2 \cdots Q_k, \qquad \widetilde{R}_k = R_k R_{k-1} \cdots R_1. \tag{2}$$

Then the relation of the QR method to power iteration comes from the fact that $\widetilde{Q}_k \widetilde{R}_k = A^k$.

Lemma 2. *In the basic QR method, $\widetilde{Q}_k \widetilde{R}_k = A^k$ for $k = 1, 2, \ldots$*

Proof. We use induction. The base case is obvious: $\widetilde{Q}_1 \widetilde{R}_1 = Q_1 R_1 = A_1 = A = A^1$ because $Q_1 R_1$ is defined to be a QR decomposition of $A_1 = A$. Suppose that $\widetilde{Q}_k \widetilde{R}_k = A^k$ for some $k \geq 1$. Then

$$\widetilde{Q}_{k+1} \widetilde{R}_{k+1} = \widetilde{Q}_k Q_{k+1} R_{k+1} \widetilde{R}_k = \widetilde{Q}_k A_{k+1} \widetilde{R}_k$$

because $A_{k+1} = Q_{k+1} R_{k+1}$ by the definition of Q_{k+1} and R_{k+1}. On the other hand, $A_{k+1} = Q_k^* A_k Q_k$ by construction, so we can iterate to obtain

$$A_{k+1} = Q_k^* Q_{k-1}^* \cdots Q_1^* A_1 Q_1 Q_2 \cdots Q_k = \widetilde{Q}_k^* A \widetilde{Q}_k.$$

Combining this with the first equation, we have

$$\widetilde{Q}_{k+1} \widetilde{R}_{k+1} = \widetilde{Q}_k \widetilde{Q}_k^* A_1 \widetilde{Q}_k \widetilde{R}_k = A \widetilde{Q}_k \widetilde{R}_k = A A^k = A^{k+1}$$

because $\widetilde{Q}_k$ is a product of unitary matrices and is therefore also unitary. This completes the proof by induction. □

To make use of this result, we use the assumption of separation of eigenvalues to diagonalize A: since all eigenvalues of A are simple, it follows that A is diagonalizable, that is, there exists a nonsingular matrix X and a diagonal matrix D such that $A = XDX^{-1}$. We can choose D such that

$$D = \begin{bmatrix} \lambda_1 & & & \\ & \lambda_2 & & \\ & & \ddots & \\ & & & \lambda_n \end{bmatrix}.$$

See section 7.8 on Jordan decomposition in Meyer's book [2]. The key consequence of this decomposition is that $A^k = XD^kX^{-1}$ is substantially easier to analyze than A^k.

Let $X = Q_X R_X$ be a QR decomposition of X. If we can argue that $\widetilde{Q}_k \to Q_X$ (which Lemma 2 will allow us to do), then we would obtain

$$A_k = \widetilde{Q}_k^* A \widetilde{Q}_k = \widetilde{Q}_k^* XDX^{-1}\widetilde{Q}_k = \widetilde{Q}_k^* Q_X R_X D R_X^{-1} Q_X^* \widetilde{Q}_k \to R_X D R_X^{-1}$$

as $k \to \infty$. Since R_X, R_X^{-1}, and D are all upper-triangular, then so, too, is $R_X D R_X^{-1}$; therefore, A_k approaches an upper-triangular matrix that is unitarily similar to A, as desired.

Unfortunately, proving that $\widetilde{Q}_k \to Q_X$ is complicated by the fact that QR decompositions are not unique; however, this is not an essential problem, as QR decompositions are unique up to a unitary diagonal factor by Lemma 1. Hence, if we can show that $\widetilde{Q}_k$ approaches the unitary factor in a QR decomposition of X, then we can argue that $\widetilde{Q}_k \to Q_X M$, where M is diagonal and unitary. Then we apply a similar argument to show that A_k approaches an upper-triangular matrix:

$$A_k = \widetilde{Q}_k^* Q_X R_X D R_X^{-1} Q_X^* \widetilde{Q}_k \to M^* Q_X^* Q_X R_X D R_X^{-1} Q_X^* Q_X M = M^* R_X D R_X^{-1} M$$

as $k \to \infty$, where the last product is upper-triangular because M, M^* and D are diagonal, and R_X and R_X^{-1} are upper-triangular. Finally, we obtain the convergence theorem for the basic QR method (in the simplest case of separated eigenvalues), due originally to Wilkinson [3]. For this proof we also assume that A is nonsingular, that is, has no zero eigenvalues.

Theorem 2. *Let $A \in \mathbb{C}^{n\times n}$ have separated, nonzero eigenvalues. Let A_k be the kth QR method iteration for A. Then A_k approaches an upper-triangular matrix that is unitarily similar to A as $k \to \infty$.*

Proof. Since the eigenvalues of A are simple, we can write $A = XDX^{-1}$, where D is a diagonal matrix whose diagonal entries are the eigenvalues of A.

Let $Y = X^{-1}$. There exists a permutation matrix Π such that ΠY admits an LU factorization. Then there exists L_Y and R_Y which are lower- and upper-triangular such that $\Pi Y = L_Y R_Y$.

Since Π is a permutation matrix, it is invertible, and so is $X\Pi^T$. Let $X\Pi^T = Q_X R_X$ be a QR decomposition of $X\Pi^T$.

By Lemma 2 and the fact that $\Pi^{-1} = \Pi^T$ because Π is a permutation, we have

$$\begin{aligned} \widetilde{Q}_k \widetilde{R}_k = A^k &= XD^kX^{-1} = X\Pi^T \Pi D^k \Pi^T \Pi Y = Q_X R_X \Pi D^k \Pi^T L_Y \Pi D^{-k} \Pi^T \Pi D^k \Pi^T R_Y \\ &= Q_X R_X (I + F_k) \Pi D^k \Pi^T R_Y, \end{aligned}$$

where

$$F_k = \Pi D^k \Pi^T L_Y \Pi D^{-k} \Pi^T.$$

Define $\Delta = \Pi D \Pi^T$. Then $F_k = \Delta^k L_Y \Delta^{-k} - I$ because $\Pi^T = \Pi^{-1}$. Moreover, since Π is a permutation, Δ is still diagonal, with the diagonal entries being a permutation of the entries of D. The order of the diagonal entries can be chosen freely as long as the columns of X are correspondingly rearranged. Thus, we can

choose the order so that the entries of Δ are in order of ascending magnitude; that is, we can ensure that $\Delta_{ii} = \lambda_i$.

Furthermore, the LU decomposition can be constructed so that the diagonal entries of L_Y are all 1. In this case, we have

$$(F_k)_{j\ell} = \begin{cases} (L_Y)_{j\ell} \left(\frac{\lambda_j}{\lambda_\ell}\right)^k & j > \ell \\ 0 & j \leq \ell. \end{cases}$$

Since the eigenvalues are separated and ordered so that $|\lambda_j| < |\lambda_\ell|$ if $j > \ell$, it follows that $(F_k)_{j\ell} \to 0$ as $k \to \infty$. Then $F_k \to 0$ as $k \to \infty$. Hence,

$$R_X(I + F_k) = (I + R_X F_k R_X^{-1}) R_X = (I + G_k) R_X,$$

where $G_k = R_X F_k R_X^{-1} \to 0$ as $k \to \infty$. For k sufficiently small, the matrix $I + G_k$ is nonsingular, so it has a QR decomposition $I + G_k = Q_G^{(k)} R_G^{(k)}$, where $Q_G^{(k)} \to I$ and $R_G^{(k)} \to I$ because $I + G_k \to I$. Note that this is only true up to multiplication by a unitary, diagonal matrix by Lemma 1; however, here we are free to choose whatever QR decomposition we want. Lemma 1 implies that we can choose QR decompositions such that $R_G^{(k)}$ has only real, positive diagonal elements. In this case, the required convergence to identity matrices is achieved.

We finally arrive at

$$\widetilde{Q}_k \widetilde{R}_k = \left(Q_X Q_G^{(k)}\right) \left(R_G^{(k)} \Delta^k R_Y\right).$$

The factors grouped on the left multiply to a unitary matrix, and the factors grouped on the right multiply to an upper-triangular matrix. Hence, the left and right sides of the above equation are both QR decompositions of the same matrix. By Lemma 2, there exists a diagonal, unitary matrix M such that $\widetilde{Q}_k = Q_X Q_G^{(k)} M$. Then $\widetilde{Q}_k \to Q_X M$ as $k \to \infty$ because $Q_G^{(k)} \to I$ as $k \to \infty$.

The proof is completed by applying the arguments discussed directly preceding this theorem, noting that $\Delta = \Pi D \Pi^T$ is still diagonal because Π is a permutation. □

Corollary 1. *The convergence of A_k to an upper triangular matrix is $\mathcal{O}(m(k))$, where*

$$m(k) = \max_{j>\ell} \left|\frac{\lambda_j}{\lambda_\ell}\right|^k.$$

Proof. The convergence of F_k to 0 in the proof above is clearly $\mathcal{O}(m(k))$. The rest of the convergence arguments involve multiplying F_k by fixed matrices, so the overall convergence is $\mathcal{O}(m(k))$. □

Remark 4. This theorem and corollary justify the stopping criterion used in Algorithm 2. Since the elements of A_k are all converging at the same rate, and we know that the elements below the diagonal are converging to 0, it makes sense to check the magnitude of these elements as a stopping condition.

2.3 The QR method with origin shifts

The obvious defect of the basic QR method is the requirement of separated eigenvalues. Not only does this mean that the method may not work in the sense of Theorem 2, but the convergence rate from Corollary 1 may be arbitrarily close to 1 even if the eigenvalues of A are separated. If A happens to be a real matrix, then it is fairly likely that A has complex eigenvalues that come in conjugate pairs; in this case A certainly does not have separated eigenvalues. Nevertheless, it can be shown that the QR method actually converges to a block upper-triangular matrix even when eigenvalues are not separated [3], and with the right care it is possible to adjust the QR method to handle this case. For example, the *double QR method* can handle real matrices with non-separated eigenvalues [1].

The best approach to dealing with convergence issues, however, is the method of *origin shifts*. The origin shift method is related to inverse power iteration in a similar manner to how the basic QR method is related to power iteration. The basic idea is to introduce a sequence of scalar *origin shifts* $\{s_k\}_{k=1}^{\infty}$. Then one step of the QR method with origin shifts is given by

$$Q_k R_k = A_k - s_k I, \qquad A_{k+1} = R_k Q_k + s_k I,$$

where $Q_k R_k = A_k - s_k I$ is a QR decomposition of $A_k - s_k I$. Note that this is still a unitary similarity transformation on A_k.

Theorem 3. *One step of the QR method with origin shifts is a unitary similarity transformation, with*

$$A_{k+1} = Q_k^* A_k Q_k.$$

Proof. We calculate

$$Q_k A_{k+1} Q_k^* = Q_k R_k Q_k Q_k^* + Q_k s_k I Q_k^* = Q_k R_k + s_k I = A_k.$$

Then $A_{k+1} = Q_k^* A_k Q_k$. □

Thus, we still end up performing a sequence of unitary similarity transformations, preserving the good numerical stability of the basic QR method; the difference from the basic method is that we get a different sequence of Q_k and R_k matrices. The key observation is that if s_k is chosen to be close to an eigenvalue λ of A, then the convergence of the QR method with origin shifts is much faster than that of the basic method, at least for the eigenvalues λ; although, the proof of this is beyond the scope of this project. Since convergence is only enhanced for one eigenvalue, this method is often used with explicit deflation, in which the QR iteration is only applied to a submatrix once convergence to one eigenvalue is achieved.

We remark that one simple choice of s_k, called the *Rayleigh quotient* [1], is $s_k = [A_k]_{nn}$, the bottom-right entry of A_k. We already expect from the theory of the basic QR method that $[A_k]_{nn}$ is converging to an eigenvalue of A, so it makes sense to use this value for s_k. Moreover, once the last row of A_k is sufficiently small (other than $[A_k]_n n$), the matrix can be deflated to the upper $(n-1) \times (n-1)$ submatrix.

The algorithm for the QR method with origin shifts differs only slightly from the basic method and is given in Algorithm 3 (although it is complicated a bit if using deflation).

3 Implementing the QR Method in MATLAB

3.1 Hessenberg matrices

We recall from our homework that QR decomposition of an $n \times n$ matrix requires $\mathcal{O}(n^3)$ additions and multiplications. Thus, each step of the QR method, implemented naively, has a cost of $\mathcal{O}(n^3)$. While a cost of $\mathcal{O}(n^3)$ for the entire method is essentially unavoidable, we would like to minimize the total cost as much as possible.

Hessenberg form

One way to reduce the cost of the QR method's iteration is through the use of *Hessenberg form*, which refers to the process of reducing a given matrix to a sparser matrix through a unitary similarity transformation. By Theorem 1, this fits nicely with the problem of finding eigenvalues and eigenvectors. Furthermore, Hessenberg form is preserved by one step of the QR method (with origin shifts), and computing the QR decomposition of a matrix in Hessenberg form costs only $\mathcal{O}(n^2)$ multiplications and additions, making it perfectly suited to the QR method. We now define Hessenberg form and demonstrate the above-listed properties of matrices in Hessenberg form.

Algorithm 3: The QR Method with Origin Shifts

Input: $A \in \mathbb{C}^{n\times n}$, a matrix with separated, nonzero eigenvalues
Input: $\{s_k\}$, a sequence of scalar origin shifts (or a rule to compute s_k on step k)
Input: $\varepsilon > 0$, stopping tolerance
Output: $\lambda_1, \lambda_2, \ldots, \lambda_n \in \mathbb{C}$, approximate eigenvalues of A
Output: $v_1, v_2, \ldots, v_n \in \mathbb{C}^n$, approximate eigenvectors of A corresponding to $\lambda_1, \ldots, \lambda_n$

$A_1 \leftarrow A$;
$k \leftarrow 1$;
repeat
  Compute s_k, if needed;
  $Q_k, R_k \leftarrow \mathrm{qr}(A_k - s_k I)$; `// qr is QR decomposition`
  $A_{k+1} \leftarrow R_k Q_k + s_k I$;
  $k \leftarrow k + 1$;
until $|[A_k]_{j\ell}| < \varepsilon$ for $j > \ell$;
for $i = 1, 2, \ldots, n$ **do**
  $\lambda_i \leftarrow [A_k]_{ii}$; `// [A_k]_ii is the ith diagonal element of A`
  $w_i \leftarrow$ eigenvector of A_k corresponding to λ_i;
  $v_i \leftarrow Q_1 Q_2 \cdots Q_{k-1} w_i$; `// Back-transformation from Theorem 1`
end

Definition 2. *A matrix $A \in \mathbb{C}^{n\times n}$ is said to be in* Hessenberg form *if $A_{ij} = 0$ for $i > j+1$. That is, A has the sparsity structure*

$$A = \begin{bmatrix} \times & \times & \times & \cdots & \times & \times \\ \times & \times & \times & \cdots & \times & \times \\ & \times & \times & \cdots & \times & \times \\ & & \ddots & \ddots & \vdots & \vdots \\ & & & \times & \times & \times \\ & & & & \times & \times \end{bmatrix},$$

where $\times$ denotes a possibly nonzero entry, and blanks denote zero entries. In other words, A is upper-triangular except for the first sub-diagonal.

First, we state without proof the following fact about Hessenberg form (a proof can be found in section 5.5 of our textbook [1]).

Theorem 4. *Given a matrix $A \in \mathbb{C}^{n\times n}$, there exists a Hessenberg matrix H and a unitary matrix U such that*

$$A = UHU^*.$$

Thus, every matrix is unitarily similar to a Hessenberg matrix.

An algorithm for computing a Hessenberg form of a matrix like the above is implemented in MATLAB by the function `hess`, so we choose not stray too far from the topic of the QR method to describe it here; suffice it to say that the unitary similarity transformation is constructed using Householder reflectors in a similar way to the Householder method of computing the QR decomposition. The cost of this algorithm is $\mathcal{O}(n^3)$, and, with the optimizations to be described subsequently, it makes up the majority of the cost of the QR method.

Second, we prove the key property of Hessenberg form, namely, that it is preserved under one step of QR method iteration.

Theorem 5. *Let $A \in \mathbb{C}^{n\times n}$ be nonsingular and in Hessenberg form. Let $A = QR$ be a QR decomposition of A. Then RQ is in Hessenberg form.*

Proof. Suppose that $A = \{a_{ij}\}$, $Q = \{q_{ij}\}$ and $R = \{r_{ij}\}$. Then $A = QR$ is equivalent to

$$A_{ij} = \sum_{k=1}^{n} Q_{ik} R_{kj}, \qquad i, j \in \{1, 2, \ldots, n\}.$$

Because R is upper-triangular, we have

$$A_{ij} = \sum_{k=1}^{j} Q_{ik} R_{kj}, \qquad i, j \in \{1, 2, \ldots, n\}.$$

Take $j = 1$. Then $Q_{i1} R_{11} = A_{i1} = 0$ if $i > 1 + 1$. Hence, $Q_{i1} = 0$ if $i > 1 + 1$ because A being nonsingular implies that $R_{11} \neq 0$. Now suppose for the sake of induction that for some $2 \leq \ell \leq n$, we have $Q_{ij} = 0$ if $i > j + 1$ for all $j = 1, 2, \ldots, \ell - 1$. Then taking $j = \ell$, if $i > \ell + 1$, then

$$0 = A_{i\ell} = \sum_{k=1}^{\ell} Q_{ik} R_{k\ell} = Q_{i\ell} R_{\ell\ell}$$

by the inductive hypothesis. Then $Q_{i\ell} = 0$ because A being nonsingular implies that $R_{\ell\ell} \neq 0$. Thus, Q is in Hessenberg form. The fact that Q is in Hessenberg form implies that RQ is as well, because

$$[RQ]_{ij} = \sum_{k=1}^{n} R_{ik} Q_{kj}, \qquad i, j \in \{1, 2, \ldots, n\}.$$

We have $R_{ik} = 0$ if $k < i$ because R is upper-triangular. Then

$$[RQ]_{ij} = \sum_{k=i}^{n} R_{ik} Q_{kj}, \qquad i, j \in \{1, 2, \ldots, n\}.$$

If $i > j + 1$, then in the above sum we have $k \geq i > j + 1$, which implies that $Q_{kj} = 0$ because Q is in Hessenberg form. Hence $[RQ]_{ij} = 0$ if $i > j + 1$, so RQ is in Hessenberg form. □

Corollary 2. *If $s \in \mathbb{C}$ is given, and if $QR = A - sI$ is a QR decomposition of $A - sI$ rather than A, then $RQ + sI$ is in Hessenberg form.*

Proof. Set $A' = A - sI$. Then A' is clearly in Hessenberg form, so by the Theorem, RQ is in Hessenberg form. Then, certainly, $RQ + sI$ is also in Hessenberg form. □

Remark 5. As we did in Theorem 2, we assume for simplicity that A is nonsingular. The theorem and Corollary are in fact still true if A is nonsingular, but a more complicated argument is required to prove them. Since we are already assuming that A is nonsingular for convergence, we opt to prove only the simpler version of this theorem.

How to use Hessenberg form with the basic QR method is described briefly in Algorithm 4. A similar approach can be used when using other variations of the QR method, such as the method with origin shifts.

QR decomposition of Hessenberg matrices

The algorithm for QR decomposition of Hessenberg matrices is the same as the Householder method for general QR decompositions. The difference that reduces the cost from $\mathcal{O}(n^3)$ operations to $\mathcal{O}(n^2)$ is a result of the fact that Hessenberg matrices are *almost* upper-triangular, which causes most of the operations in the QR decomposition with the Householder method to be multiplication by 0 or addition of 0, which can be skipped.

Algorithm 4: The Basic QR Method in Hessenberg Form

Input: $A \in \mathbb{C}^{n\times n}$, a matrix with separated, nonzero eigenvalues
Output: $\lambda_1, \lambda_2, \ldots, \lambda_n \in \mathbb{C}$, approximate eigenvalues of A
Output: $v_1, v_2, \ldots, v_n \in C^n$, approximate eigenvectors of A corresponding to $\lambda_1, \lambda_2, \ldots, \lambda_n$

$H, P \leftarrow \text{hess}(A)$; // Put A in Hessenberg form: $A = PHP^*$
$\lambda_1, \lambda_2, \ldots, \lambda_n, w_1, w_2, \ldots, w_n \leftarrow \text{qr\_method}(H)$; // qr_method is Algorithm 2
for $i = 1, 2, \ldots, n$ **do**
  $v_i \leftarrow P^* w_i$; // Back-transform eigenvectors using Theorem 1
end

To be precise, the vector u in Algorithm 1 has only two nonzero components if A is in Hessenberg form, namely, the first two components. Then only the first two components of v are nonzero, and the matrix vv^* is zero everywhere except in the upper-left 2×2 submatrix. Then the update to $R^{(k)}$ and $Q^{(k)}$ only affects the first two rows, and it can be computed by ignoring all but the first two rows of $R^{(k)}$ and $Q^{(k)}$. Thus, the update is computed by a 2×2 times $2 \times n$ matrix multiplication instead of $(n-k+1) \times (n-k+1)$ times $(n-k+1) \times n$ matrix multiplication, reducing the average update cost from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$, and thereby reducing the whole cost from $\mathcal{O}(n^3)$ to $\mathcal{O}(n^2)$.

These considerations are described precisely in Algorithm 5.

Algorithm 5: QR Decomposition for Hessenberg Matrices

Input: $A \in \mathbb{C}^{n\times n}$, a matrix in Hessenberg form
Output: $Q \in \mathbb{C}^{n\times n}$, a unitary matrix
Output: $R \in \mathbb{C}^{n\times n}$, an upper-triangular matrix such that $A = QR$

$Q \leftarrow I$; // I is $n \times n$ identity
$R \leftarrow A$;
for $k = 1, 2, \ldots, n-1$ **do**
  $u \leftarrow \begin{bmatrix} R_{kk} & R_{(k+1)k} \end{bmatrix}^T$; // We only need the first two entries because A is Hessenberg
  $\alpha = \text{rot}(u_1)\|u\|$; // $\text{rot}(u_1)$ gets a unit complex number rotated 180° from u_1
  $u_1 \leftarrow u_1 - \alpha$;
  $v \leftarrow \frac{u}{\|u\|}$; // Note that v is 2×1, independent of k
  $R^{(k,2)} \leftarrow R^{(k,2)} - 2vv^* R^{(k,2)}$; // $R^{(k,2)}$ is the k and $(k+1)$th rows of R, always $2 \times n$
  $Q^{(k,2)} \leftarrow Q^{(k,2)} - 2vv^* Q^{(k,2)}$; // $Q^{(k,2)}$ is the k and $(k+1)$th rows of Q, always $2 \times n$
end
$Q \leftarrow Q^*$; // We were actually tracking Q^* through the algorithm

3.2 Implementing the algorithms

Up to this point we have described the mathematical background necessary to implement the QR method (possibly with origin shifts). All that remains now is to do the implementation, for which we use MATLAB. This implementation is mostly described by the comments in the code files themselves; however, we will outline the structure of the code and the purpose of the files in this section.

`hess`

`hess` is a built-in MATLAB command that converts a matrix to Hessenberg form through a unitary similarity transformation constructed by composing Householder reflectors, very similar to how the QR decomposition is done. The function outputs the Hessenberg form of the input matrix as well as the unitary matrix that

can be used to convert between the matrix and the Hessenberg form. We need this unitary matrix to back-transform eigenvectors through the conversion to Hessenberg form.

`hessqr.m`

`hessqr.m` contains two functions: `hessqr` and `rot`. The function `rot` gets a unit complex number that is rotated from the input number by 180° in the complex plane, which is the complex equivalent of the negation used in the algorithm for the real QR method to ensure numerical stability.

The main function, `hessqr`, implements the fast QR decomposition for Hessenberg matrices described in Algorithm 5. The implementation is straightforward translation of the algorithm from the pseudocode presented here to MATLAB.

`basicalgorithm.m`

`basicalgorithm.m` contains three functions: `basicalgorithm`, `stopping_criterion`, and `triu_eigenvectors`. The function `stopping_criterion` computes the stopping criterion in Algorithm 2, and the function `triu_eigenvectors` computes the eigenvectors of an upper-triangular matrix, which is needed after the QR iteration converges if the user requests the eigenvectors of the matrix.

The main function, `basicalgorithm`, uses the basic QR method after initial reduction to Hessenberg form to find the eigenvalues of a given matrix A. The user may also request for eigenvectors to be computed by supplying an optional output in which to store the eigenvectors. The eigenvector computation is relatively expensive compared to the eigenvalue computation and is not always needed in practice, so we made it an optional part of the function.

4 Experiments

In this section we perform experiments to test whether our implementation works and also validate some of the experimental results that we presented.

4.1 Accuracy of the QR method

We start by examining the convergence rate of the basic QR method. We construct a random matrix with eigenvalues whose magnitudes are $1, 2, \ldots, n$ as follows. First, we generate a random complex matrix B whose entries are generated by setting the real and imaginary parts equal to independent random numbers that are uniformly distributed on $[0, 1]$. Next, we take the SVD of B to obtain unitary matrices U and V and a diagonal matrix D such that $B = UDV^*$. Then we generate an upper triangular matrix T whose jth diagonal entry is given by $je^{2\pi i u_j}$, where u_j is uniformly distributed on $[0, 1]$, and the rest of whose entries are chosen in the same manner as the entries of B. Then we set $A = UTU^*$. It follows that the eigenvalues of A have magnitudes $1, 2, \ldots, n$. Note that this gives $m(k) = \left(\frac{n-1}{n}\right)^k$. We implement this experiment in `experiment1.m`.

Part 1

In this part of the experiment we verify that our implementation correctly produces the eigenvalues and eigenvectors of a random matrix A constructed as described above. We compute the absolute errors between the eigenvalues and the ℓ^∞ error between the eigenvectors (after normalizing them so that they are in the same direction and are both unit vectors). We use a brute-force matching algorithm to identify corresponding eigenvalues and eigenvectors, using the absolute difference as the distance function for matching eigenvalues and the absolute cosine similarity as the distance function for matching eigenvectors.

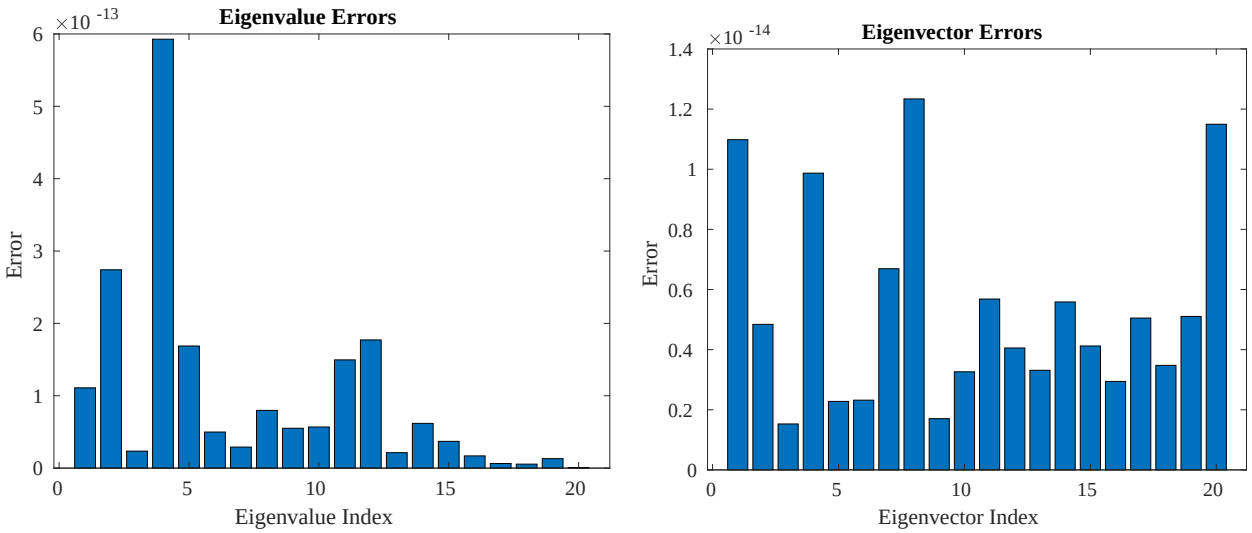

Figure 1: (Left) Absolute errors between the true eigenvalues and the eigenvalues computed using the QR method. (Right) ℓ^∞ errors between the true eigenvectors and the eigenvectors computed using the QR method.

Figure 1 shows the errors for the eigenvalues and eigenvectors using the basic QR method with stopping tolerance $\varepsilon = 10^{-12}$. We observe that all eigenvalues and eigenvector components are within roughly 10^{-12} of the true values, verifying that our implementation is correct. Furthermore, this validates the use of the stopping criterion in Algorithm 2.

Part 2

In this part we measure the stopping criterion for A_k, the maximum absolute value of the lower triangular entries of A_k, on each step k. Figure 2 shows the maximum absolute value of the lower triangular entries of A_k. We observe that the error curve becomes parallel to the reference curve $m(k)$, which represents exponential decay in k. Since the plot uses a log scale on the error axis, this implies that the convergence of the basic QR method is $\mathcal{O}(m(k))$, as predicted by Corollary 1.

4.2 Convergence failures(?)

In this section we demonstrate what happens when the QR method is applied to matrices that violate the assumptions of nonzero, separated eigenvalues. We implement this experiment in `experiment2.m`.

Part 1

We start with a simple 3×3 matrix A that has one real eigenvalue and two complex eigenvalues:

$$A = \begin{bmatrix} 2 & 3 & 4 \\ -2 & 1 & 0 \\ 1 & 2 & 0 \end{bmatrix}.$$

We run 50 steps of the basic QR method on this matrix and obtain the following matrix A_{50}:

$$A_{50} \approx \begin{bmatrix} 2.4093 & -4.5482 & -1.4324 \\ 1.4080 & 2.2739 & 0.6865 \\ 0 & 0 & -1.6832 \end{bmatrix}.$$

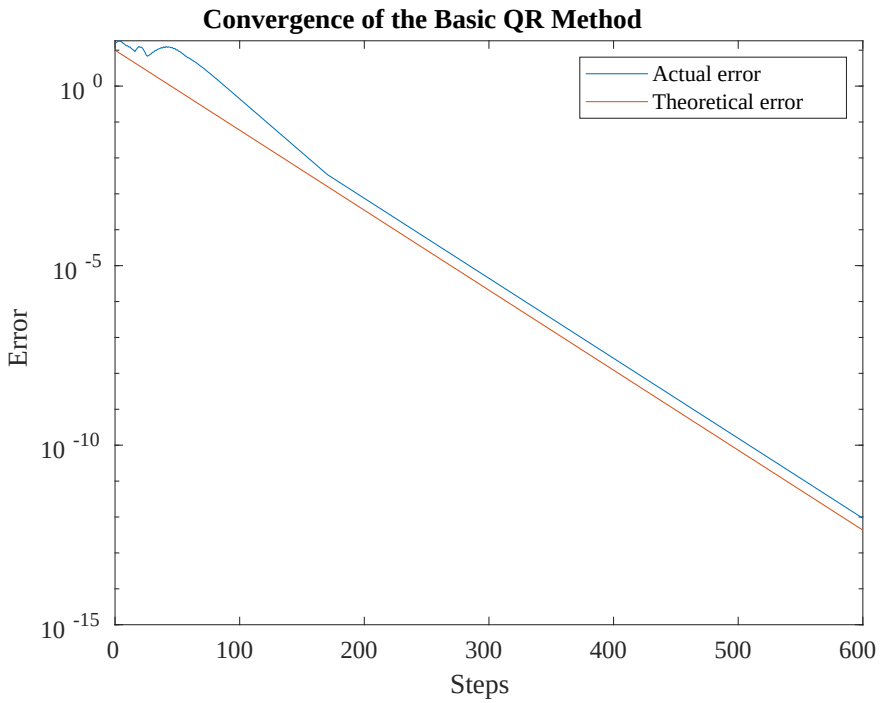

Figure 2: Convergence of the basic QR method to an upper triangular matrix. The actual maximum absolute value of the lower triangular part of the iterates and the theoretical reference error $m(k)$ are plotted for each step k.

We observe that A_{50} is *block upper triangular*. The value -1.6832 on the diagonal is very close to the real eigenvalue of A, and the eigenvalues of the upper-left 2×2 submatrix of A_{50} are very close to the complex eigenvalues of A.

Although the basic QR method is clearly not converging to an upper-triangular matrix, it does seem to be converging to a block upper triangular matrix, whose diagonal blocks can be used to find the eigenvalues of A. It is beyond the scope of this paper to prove it (even our textbook [1] does not provide a proof, just the statement), but if A is a real matrix, then the basic QR method always converges to a block upper triangular matrix[3] whose diagonal blocks are either 1×1, corresponding to real eigenvalues of A, or 2×2, whose eigenvalues are complex eigenvalue pairs of A. The double QR method mentioned earlier is designed around exploiting this fact.

Part 2

In this part we demonstrate the observations made in Part 1 on a larger scale. We generate a real, 10×10 matrix B and run 1000 steps of the basic QR method on B. For one particular run, the output of which is

[3]This is the reason for the "?" in section title. Although the QR method does not, in general, converge in the sense of Theorem 2 specifically, it *does* always converge in a more general sense.

stored in `experiment2_output.txt`, we obtain the following matrix B_{1000}:

$$B_{1000} = \begin{bmatrix} 5.0029 & -0.4810 & 0.4937 & -0.1995 & 0.3692 & -0.2716 & -0.4672 & 0.2088 & 0.1050 & 0.1001 \\ 0 & -1.0326 & -0.1319 & -0.1990 & 0.1406 & -0.1589 & -0.3451 & -0.0119 & -0.1097 & 0.0197 \\ 0 & 0 & -0.6263 & -0.3278 & 0.0980 & -0.1330 & -0.2053 & -0.2685 & 0.1224 & -0.1714 \\ 0 & 0 & 0.4937 & -0.7493 & 0.0624 & 0.5047 & -0.4036 & -0.3831 & -0.1645 & 0.1736 \\ 0 & 0 & 0 & 0 & -0.4732 & -0.8197 & 0.4169 & 0.2311 & 0.2668 & 0.0755 \\ 0 & 0 & 0 & 0 & 0.3931 & -0.2283 & -0.2024 & -0.0914 & -0.2499 & -0.2570 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.5741 & -0.1578 & -0.2442 & 0.5990 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0.0874 & 0.5616 & 0.3131 & -0.0817 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0.2517 & -0.3398 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & -0.1416 \end{bmatrix}.$$

Here we have indicated 1×1 diagonal blocks in red and 2×2 diagonal blocks in blue. Checking the eigenvalues of these diagonal blocks against the true eigenvalues of B verifies the observations in Part 1 (see `experiment2_output.txt`).

5 Conclusion

In this project I learned about the QR method for finding all the eigenvalues and eigenvectors of a matrix A, which is an important problem, as eigenvalues and eigenvectors are useful in a wide range of applications. Indeed, the QR method is widely used in practical applications [1]; however, I mostly studied the basic QR method, whose convergence is too slow to be useful in many cases. Additionally, the basic method only applies to matrices with separated eigenvalues. As mentioned, using origin shifts with deflation is the primary means by which the QR method is accelerated and applied to general matrices.

I derived the convergence rate of the basic QR method and conducted experiments demonstrating this convergence. I also demonstrated through my experiments why the assumption of separated eigenvalues is necessary for the QR method to converge; although, I also demonstrated that the convergence failure is not so bad, as QR method does still converge to *something* when the eigenvalues of A are not separated, and the eigenvalues and eigenvectors of A can still be recovered from that *something*. This shows how, with the proper modifications, the QR method can be one of the most general and powerful approaches to finding eigenvalues and eigenvectors numerically.

References

[1] Ackleh, A. S., Allen, E. J., Kearfott, R. B., and Seshaiyer, P. *Classical and Modern Numerical Analysis*. Chapman and Hall/CRC, 2009.

[2] Meyer, C. D. *Matrix Analysis and Applied Linear Algebra*. Society for Industrial and Applied Mathematics, Philadelphia, 2008.

[3] Wilkinson, J. H. Convergence of the LR, QR, and Related Algorithms. *The Computer Journal*, 8(1):77–84, 1965.